
Revisiting Efficiency–Accuracy Scaling in Mixture-of-Experts Architectures

Venmugil Elango¹ Nidhi Bhatia¹ Roger Waleffe¹ Rasoul Shafipour¹ Tomer Asida¹ Abhinav Khattar¹
Nave Assaf¹ Maximilian Golub¹ Joseph Guman¹ Tiya Mitra¹ Ritchie Zhao¹ Ritika Borkar¹
Ran Zilberstein¹ Mostofa Patwary¹ Mohammad Shoeibi¹ Bitu Darvish Rouhani¹

Abstract

Mixture-of-Experts (MoEs) have become a central component of many state-of-the-art open-source and proprietary large language models. Despite their widespread adoption, it remains unclear how close existing MoE architectures are to optimal for inference cost, as measured by accuracy per floating-point operation and per parameter. In this work, we revisit MoE design from a hardware–software co-design perspective, grounded in empirical and theoretical considerations. We characterize key performance bottlenecks across diverse deployment regimes, spanning offline high-throughput execution and online, latency-critical inference. Guided by these insights, we introduce **LatentMoE**, a new model architecture resulting from systematic design exploration and optimized for maximal accuracy per unit of compute. Empirical design space exploration at scales of up to 95B parameters and over a 1T-token training horizon, together with supporting theoretical analysis, shows that LatentMoE consistently outperforms standard MoE architectures in terms of accuracy per FLOP and per parameter. Given its strong performance, the LatentMoE architecture has been adopted by the flagship Nemotron-3 Super and Ultra models and scaled to substantially larger regimes, including longer token horizons and larger model sizes, as reported in (NVIDIA et al., 2025).

1. Introduction

Transformer-based large language models underpin a wide range of modern AI systems, from conversational agents to code generation and scientific reasoning. As these models

continue to scale, practical deployment is increasingly constrained by inference cost, encompassing both computation and memory. As a result, a central objective in modern model design is to maximize achievable accuracy under fixed inference cost constraints.

Mixture-of-Experts (MoE) architectures have emerged as a promising approach towards this goal, enabling models to scale parameter count while keeping the number of Floating-point Operations (FLOPs) per token fixed. Despite their empirical success, the MoE design space remains poorly understood. Existing MoE architectures are largely motivated by high-level sparsity arguments and optimized for offline, throughput-oriented settings. These designs give limited consideration to online deployments, which impose strict latency, memory bandwidth, and communication constraints.

Effective MoE design must be evaluated along two complementary dimensions: accuracy per FLOP and accuracy per parameter. While accuracy per FLOP captures computational efficiency, accuracy per parameter reflects memory footprint, memory bandwidth demands, routing-induced communication, and sharding overheads (factors that are often the dominant bottlenecks in interactive, low-latency inference). Neglecting these factors can lead to architectures that appear efficient in aggregate compute, yet incur substantial inefficiencies in practical deployment.

We revisit MoE architecture design from a hardware–software co-design perspective. Through a systematic analysis of existing MoE systems across the throughput–latency Pareto frontier, we identify key structural bottlenecks arising from expert parameterization, routing-induced all-to-all communication, and memory access patterns. Combined with detailed accuracy measurements and theoretical analysis, our study identifies structural inefficiencies in prevailing MoE designs that limit achievable accuracy per unit of inference cost.

Guided by these insights, we introduce LatentMoE, a new mixture-of-experts architecture explicitly optimized for both accuracy per FLOP and accuracy per parameter. LatentMoE decouples expert routing and computation from the model hidden dimension by projecting incoming activations into a shared low-dimensional latent space prior to expert

¹NVIDIA Corporation, Santa Clara, California, USA. Correspondence to: Venmugil Elango <velango@nvidia.com>, Nidhi Bhatia <nidhib@nvidia.com>.

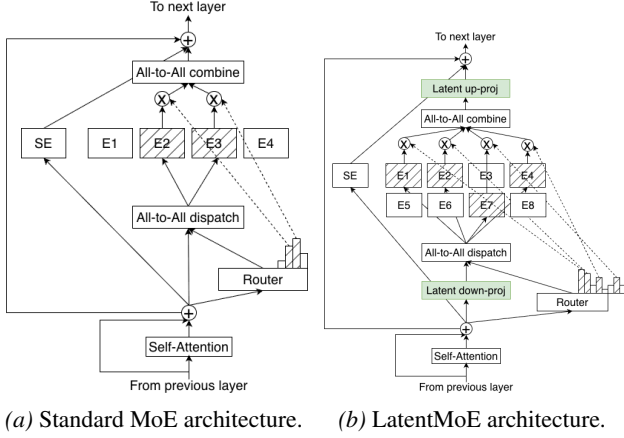


Figure 1. Standard MoE vs. LatentMoE architectures. In LatentMoE, tokens are projected from the model hidden dimension d into a smaller latent dimension ℓ for expert routing and computation, which reduces routed parameter loads and all-to-all traffic by a factor of d/ℓ . We use this efficiency to increase the total number of experts and the top- K active experts per token by the same factor d/ℓ , which improves the accuracy of the model while keeping overall inference cost approximately constant.

processing (Figure 1). The latent dimension serves as a direct control knob for computational cost, communication volume, and expert parameter size. At iso-FLOP and iso-parameter count, projecting incoming activations into a lower-dimensional latent space enables a proportional increase in both the number of experts and the routing top- K , while maintaining constant inference cost.

As we show both theoretically and empirically, simultaneously increasing expert count and combinatorial sparsity diversity improves the effective expressivity of the model, leading to higher achievable accuracy. Crucially, these gains arise without increasing memory bandwidth demands or communication overheads, making LatentMoE well suited for both latency-critical and throughput-oriented deployments. We validate the LatentMoE concept through pretraining experiments at scales of up to 95B parameters and over 1T tokens. Across all evaluated regimes, LatentMoE consistently improves upon standard MoE architectures, achieving higher accuracy at fixed inference cost or substantially lower inference cost at fixed accuracy.

2. LatentMoE Core Design Principles

Before delving into the specifics of LatentMoE, we first take a systems-level view of what is required to deploy an MoE model that is both accurate and cost-efficient. Table 1 summarizes the notation.

Throughout this section, we use Qwen3-235B-A22B as a running example for our modeling, with $N = 128$ experts, $K = 8$ active experts per token, a hidden di-

Table 1. Notation used throughout the paper.

d	Model hidden dim.	t_{exp}	Tokens per expert
ℓ	LatentMoE latent dim.	t_{total}	Total tokens / MoE layer
m	FFN intermediate dim.	α	Compression ratio d/ℓ
N	Total routed experts	r_{eff}	Intrinsic feature rank
K	Active experts / token	F	Peak FLOP/s per GPU
S	Shared experts	BW_{NVL}	NVLink bandwidth
EP	Expert-parallel degree	BW_{HBM}	HBM bandwidth

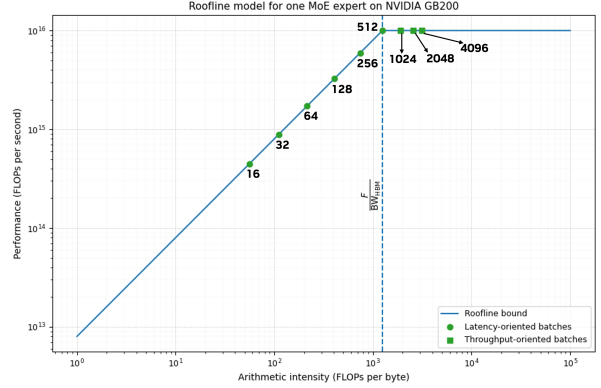


Figure 2. Roofline Analysis for serving Qwen3-235B-A22B. Operating points correspond to per-expert token counts t_{exp} (i.e., effective expert batch sizes after MoE routing), mapped to arithmetic intensity I . At latency-critical batch sizes (low I), expert computation is constrained by HBM bandwidth rather than compute, and the operating points lie in the bandwidth-bound regime.

mension $d = 4096$, and an intermediate feed-forward dimension $m = 1536$. For concreteness, we consider deployment on NVIDIA GB200 GPUs interconnected by a high-bandwidth NVLink fabric, which provides approximately 1800 GB/s of bidirectional bandwidth per GPU (i.e., $\text{BW}_{\text{NVL}} = 900$ GB/s per direction). To ensure that expert communication remains within a single NVLink domain, experts are distributed via expert parallelism across $\text{EP} = 64$ GPUs. Attention layers are executed using data parallelism over the same group of GPUs. Each GB200 GPU delivers a peak FP4 Tensor Core throughput of $F = 10$ PFLOPs and an HBM memory bandwidth of $\text{BW}_{\text{HBM}} = 8$ TB/s (NVIDIA Corporation, 2024).

2.1. Memory Bandwidth Bottleneck

In highly interactive (i.e., low-latency) settings that typically use small batch sizes, MoE computation is primarily bottlenecked by memory bandwidth. Figure 2 provides a high-level roof-line analysis of performance versus arithmetic intensity.

For GB200, a computation becomes compute-bound only if its arithmetic intensity (i.e., FLOPs per byte) exceeds:

$$\frac{F}{\text{BW}_{\text{HBM}}} = \frac{10 \times 10^{15}}{8 \times 10^{12}} = 1250 \text{ FLOPs/byte.}$$

Let t_{total} be the total number of tokens across the EP GPUs prior to MoE routing. Assuming a uniform distribution of tokens across experts, the number of tokens assigned to a single expert is: $t_{\text{exp}} := \frac{t_{\text{total}} \cdot K}{N}$. In the Qwen3-235B-A22B example, with $N = 128$ and $\text{EP} = 64$, each GPU hosts $N/\text{EP} = 2$ experts; thus each GPU processes approximately $2 \cdot t_{\text{exp}}$ expert tokens per MoE layer.

The FP4 compute cost for a single matrix multiplication within an expert (*i.e.*, either FC1 or FC2) is $C_{\text{exp}} = 2 \cdot t_{\text{exp}} \cdot d \cdot m$, and the corresponding memory traffic in FP4 precision—accounting for weights, inputs, and outputs—is given by $M_{\text{exp}} = 0.5625 \cdot [d \cdot m + t_{\text{exp}} \cdot (d + m)]$ bytes, where 0.5625 bytes/element reflects the 4-bit value (0.5 B) plus an FP8 microblock scale shared across 16 elements (0.0625 B/element). Since the GEMMs across both local experts share approximately the same shape, the per-GPU arithmetic intensity I matches that of a single GEMM:

$$I = \frac{C_{\text{exp}}}{M_{\text{exp}}} = \frac{2 \cdot t_{\text{exp}} \cdot d \cdot m}{0.5625 \cdot [d \cdot m + t_{\text{exp}} \cdot (d + m)]}.$$

To operate in the compute-bound regime, we require $I \geq 1250$. Substituting the Qwen3-235B-A22B parameters yields the threshold $t_{\text{exp}} \geq 514$. In typical latency-critical deployments, effective batch sizes are well below 100 tokens—comfortably below the threshold of 514. Consequently, MoE experts operate in the memory-bound region of the roofline curve (Figure 2), where performance is limited by weight loading rather than compute capacity.

Design Principle I: In low-latency serving scenarios, MoE inference is typically dominated by the memory-bandwidth cost of loading model weights. Consequently, maximizing accuracy per parameter is critical for applications with high interactivity requirements.

2.2. Communication Bottleneck

In throughput-oriented settings, once experts become compute-bound, communication emerges as a significant contributor to end-to-end execution time in distributed settings. Expert parallelism mandates all-to-all token routing across devices, imposing an overhead that can dominate end-to-end execution time. Recall that in our example configuration, each GPU hosts two experts, so each GPU processes a total of $2t_{\text{exp}}$ tokens across its local experts per MoE layer.

Assuming a uniform distribution of tokens, the all-to-all communication volume per GPU per MoE layer is given by:

$$M_{\text{comm}} = 2.5625 \cdot \left(\frac{N}{\text{EP}} \cdot t_{\text{exp}} \cdot d \right) = 5.125 \cdot t_{\text{exp}} \cdot d.$$

Here, the factor 2.5625 accounts for the mixed-precision traffic (0.5625 bytes for FP4 dispatch including the microblock scale, 2 bytes for BF16 aggregation). On the com-

pute side, the per-layer FLOP count across both matmuls (FC1 and FC2) of the two local experts is:

$$C_{\text{comp}} = 4 \cdot \left(\frac{N}{\text{EP}} \cdot t_{\text{exp}} \cdot d \cdot m \right) = 8 \cdot t_{\text{exp}} \cdot d \cdot m,$$

where the factor 4 combines 2 FLOPs/MAC with 2 matmuls per expert (FC1 and FC2). The corresponding compute time is $t_{\text{comp}} = \frac{C_{\text{comp}}}{F} = \frac{8 \cdot t_{\text{exp}} \cdot d \cdot m}{F}$. Similarly, the all-to-all communication time is: $t_{\text{comm}} = \frac{M_{\text{comm}}}{\text{BW}_{\text{NVL}}} = \frac{5.125 \cdot t_{\text{exp}} \cdot d}{\text{BW}_{\text{NVL}}}$. Consequently, the ratio of communication time to compute time is:

$$\frac{t_{\text{comm}}}{t_{\text{comp}}} = \frac{5.125 \cdot t_{\text{exp}} \cdot d / \text{BW}_{\text{NVL}}}{8 \cdot t_{\text{exp}} \cdot d \cdot m / F} = \frac{5.125 \cdot F}{8 \cdot m \cdot \text{BW}_{\text{NVL}}}.$$

Substituting the parameters for the GB200 NVL72 and Qwen3-235B-A22B yields a ratio of approximately 4.6. This indicates that in the throughput-oriented regime, MoE layers are dominated by all-to-all communication overhead.

Design Principle II: Improving performance in throughput-oriented MoE deployments requires minimizing the data volume of all-to-all operations. This volume:

$$M_{\text{comm}} \propto \frac{N}{\text{EP}} \cdot t_{\text{exp}} \cdot d = \frac{t_{\text{total}} \cdot K \cdot d}{\text{EP}}.$$

Consequently, communication overhead can be mitigated by reducing the routed hidden dimension d or the number of active experts K . Note that modifying the intermediate dimension m does not affect the token size and thus yields no direct improvement.

2.3. Model Quality

Beyond optimizing inference speed, preserving model quality is paramount. To this end, we draw on theoretical insights into neural network expressivity and combinatorial sparsity. Classical results on Barron functions (Barron, 1993) state that a one-hidden-layer network with u nonlinear units achieves a mean-squared error of $\mathcal{O}(1/u)$, independent of the input dimension d . In an MoE layer, this effective nonlinear budget per token is proportional to the total width of the selected experts, $U_{\text{eff}} \propto K \cdot m$. This implies that reducing the active experts K or the intermediate dimension m directly penalizes the effective capacity (U_{eff}), risking model quality degradation.

Design Principle III: Maintaining model quality requires preserving the effective nonlinear budget, $K \cdot m$. Consequently, to alleviate memory and communication bottlenecks without sacrificing model quality, we should keep both the number of active experts and the intermediate dimension unchanged.

Every inference task is characterized by an intrinsic feature rank, r_{eff} , corresponding to the minimum number of

degrees of freedom required to preserve task-relevant information. Reducing the hidden dimension d below this threshold necessarily discards such information, leading to accuracy degradation. Thus, r_{eff} serves as a task-dependent lower bound on d .

Design Principle IV: There exists a task-specific feature rank r_{eff} that imposes a lower limit on the reduction of d . Reducing d below this limit precipitates a model collapse.

Additionally, the MoE architecture benefits from combinatorial sparsity, offering $\binom{N}{K}$ possible expert combinations per token (Dai et al., 2024). Increasing the total number of experts N expands this specialization space. Furthermore, scaling both N and K by a factor α exponentially increases the diversity of expert mixtures:

$$\binom{\alpha N}{\alpha K} \geq \left(\binom{N}{K} \right)^\alpha.$$

Design Principle V: Scaling both the number of experts N and top- K per token enhances model quality by exponentially expanding the space of expert combinations.

Putting it all together. Design Principles I and II indicate that improving inference speed requires reducing both memory bandwidth and communication costs. Memory bandwidth cost scales with d and m , while communication cost scales with K and d . However, Principle III cautions against reducing either K or m , as doing so would likely degrade model quality. This leaves d as the most promising dimension to reduce, enabling performance improvements in both throughput- and latency-oriented regimes without significant loss in accuracy. Principle IV further establishes a lower bound, (r_{eff}), on d to prevent quality collapse. Moreover, Principle V suggests that increasing N and K improves model quality. Since memory bandwidth and communication costs scale linearly with K , we can simultaneously increase K by a factor α and reduce d by the same factor α (provided $d/\alpha \geq r_{\text{eff}}$). We hypothesize, and empirically validate in subsequent sections, that this transformation (also depicted in Figure 1b) preserves memory bandwidth and communication costs while improving network expressivity and combinatorial sparsity, yielding higher accuracy per FLOP and per parameter.

3. LatentMoE Architecture

Guided by the design principles outlined in Section 2, we introduce LatentMoE, a new MoE architecture designed for efficient scaling. LatentMoE first projects each input token $x \in \mathbb{R}^d$ into a lower-dimensional latent space $\mathbb{R}^\ell$ using a learnable down-projection matrix $W_\downarrow \in \mathbb{R}^{\ell \times d}$. The compressed representation is then routed to the selected experts. Each expert $E_i(\cdot; \ell)$ operates entirely within the latent space

and is parameterized by weights $W_{\text{FC1}}^{(i)}, W_{\text{gate}}^{(i)} \in \mathbb{R}^{m \times \ell}$ and $W_{\text{FC2}}^{(i)} \in \mathbb{R}^{\ell \times m}$. After expert computation, the outputs are aggregated and projected back to the original input dimension using a learnable up-projection matrix $W_\uparrow \in \mathbb{R}^{d \times \ell}$.

Since we compress only the input dimension d to ℓ while keeping the intermediate dimension m constant, the effective nonlinear budget U_{eff} remains unchanged. While Design Principle III suggests this should theoretically preserve accuracy, in practice, larger models are often easier to train and more robust to hyperparameter variations (Frankle & Carbin, 2019; Novak et al., 2018; Taylor et al., 2021). To avoid extensive hyperparameter tuning for the compressed model, we leverage Design Principle V by scaling the total number of experts N by a factor α , thereby expanding the combinatorial specialization space. Crucially, since neither the memory bandwidth cost (in latency-oriented scenarios) nor the communication cost (in throughput-oriented scenarios) depends on N , this scaling adheres to Design Principles I and II, incurring no additional inference overhead. We refer to this modification as ℓ -MoE_{eff}, defined as follows:

$$\ell\text{-MoE}_{\text{eff}}(x) := W_\uparrow \cdot \left(\sum_{i \in \mathcal{T}_{K, N'}} p'_i E_i(W_\downarrow \cdot x; \ell) \right) + \sum_{j=N'+1}^{N'+S} E_j(x; d). \quad (1)$$

Here, $N' = \alpha \cdot N$ denotes the expanded set of routed experts. The routed experts $E_i(\cdot; \ell)$ operate in the latent space and the shared experts $E_j(\cdot; d)$ in the original space. The routing weights $p' = \text{Softmax}(W_r' \cdot x)$ are computed from token $x \in \mathbb{R}^d$ via a learnable matrix $W_r' \in \mathbb{R}^{N' \times d}$, and $\mathcal{T}_{K, N'}$ denotes the top- K experts (out of N') selected by routing score. For simplicity, all operations outside the routed experts—including the MoE routing and shared experts—continue to operate in the original hidden dimension d , as they do not significantly contribute to the memory and communication bottlenecks.

Following the down-projection $W_\downarrow$, token dispatch and aggregation occur in the latent space $\mathbb{R}^\ell$. This reduces the communication volume by a factor of α relative to a standard MoE. Similarly, because the expert weights lie in the latent space ($\mathbb{R}^{m \times \ell}$ and $\mathbb{R}^{\ell \times m}$), the memory bandwidth cost for weight loading is also reduced by a factor of α .

Design Principle V further suggests that scaling both N and K by a factor α exponentially increases expert diversity, thereby enhancing model quality. Following this principle, the default LatentMoE configuration (a.k.a., ℓ -MoE_{acc}) is

Table 2. Comparison of asymptotic communication and memory bandwidth costs per GPU. Costs are normalized by hardware constants. Arrows indicate improvement ($\uparrow$), maintenance ($\rightarrow$), or baseline ($-$).

Architecture	Communication Cost	Weight Loading Memory Cost per Expert	Model Accuracy	Inference Efficiency
Standard MoE	$(N/EP) \cdot t_{\text{exp}} \cdot d$	$d \cdot m$	$-$	$-$
ℓ -MoE _{eff}	$(N/EP) \cdot t_{\text{exp}} \cdot \ell$	$\ell \cdot m$	$\rightarrow$	$\uparrow$
ℓ -MoE _{acc} (recommended)	$(N/EP) \cdot t_{\text{exp}} \cdot d$	$d \cdot m$	$\uparrow$	$\rightarrow$

defined as follows:

$$\ell\text{-MoE}_{\text{acc}}(x) := W_{\uparrow} \cdot \left(\sum_{i \in \mathcal{T}_{K', N'}} p'_i E_i(W_{\downarrow} \cdot x; \ell) \right) + \sum_{j=N'+1}^{N'+S} E_j(x; d), \quad (2)$$

where $K' = \alpha \cdot K$. This formulation differs from ℓ -MoE_{eff} solely in the number of active experts, utilizing the top- K' selection function $\mathcal{T}_{K', N'}$.

Since K is increased by $\alpha = d/\ell$, this variant keeps communication and memory bandwidth costs constant relative to a standard MoE. The increased expert diversity and per-token nonlinearity budget, however, yield superior accuracy at iso-inference cost, pushing the Pareto frontier to a new level. Table 2 summarizes the costs and benefits of ℓ -MoE_{eff} and ℓ -MoE_{acc}. For completeness, we evaluate both setups in Section 4.

4. Evaluation

In this section, we conduct a thorough design space exploration to verify the effectiveness of the proposed LatentMoE architecture. We start by pretraining Transformer MoE models at two different scales: (1) 16B total parameters with 2B active, which we use for conducting ablation studies, and (2) 95B total parameters with 8B active, which we use as a scaling test of the 16B results. To demonstrate the generalizability of LatentMoE, we further extend our study by training hybrid Mamba-Attention MoE models at scale.

We use the architecture and hyperparameters from DeepSeek-v2-lite (DeepSeek-AI et al., 2024) for our 2B active model ablations. For the 8B active Transformer model, we use a cosine learning rate schedule with a max learning rate of 1.2×10^{-3} decayed to a minimum of 3×10^{-6} . The 8B active Hybrid model is trained with a WSD schedule with a max learning rate of 8×10^{-4} decayed to 8×10^{-6} in the last 15% of training. Both the 8B active Transformer and hybrid models are trained with a sequence length of 8192, a batch size of 768 (≈ 6 M tokens), and a learning rate warmup of 8.4B tokens. The 8B active models use a load balancing loss coefficient of 10^{-4} along with DeepSeek’s

Table 3. Architectural specifications of the baseline models used for design space exploration. For the Hybrid model’s Mamba layers, we use Mamba-2 blocks with 128 heads, 64 head dimension, 128 state dimension, and 8 groups.

Configuration	16BT-2BA	95BT-8BA	Hybrid-73BT-8BA
Layers (L)	27	32	52 (24 Mamba/MoE, 4 Attn.)
Hidden Dimension (d)	2048	4096	4096
Total Routed Experts (N)	64	128	128
Active Experts (K)	6	6	6
Shared Experts (S)	2	2	2
Intermediate FFN Dimension (m)	1408	2688	2688
Activation Function	SwiGLU	Squared-ReLU	Squared-ReLU
Attention Heads	16	32	32
Query Groups (GQA)	16	8	8
Total Parameters	16B	95B	73B
Active Parameters	2B	8B	8B

aux-loss-free load balancing strategy (Wang et al., 2024) to ensure balanced token load throughout training. Table 3 summarizes the model architecture under study in this paper.

4.1. LatentMoE Ablations

Impact of compression ratio. Design Principle IV (Section 2) hypothesizes that there exists an intrinsic rank r_{eff} such that compressing the latent dimension to $\ell \geq r_{\text{eff}}$ results in negligible information loss. To empirically validate this and estimate r_{eff} , we pretrain and sweep different compression ratios on top of the ℓ -MoE_{eff} configuration, holding all other hyperparameters constant.

Results in Figure 3 indicate that model quality is preserved for compression ratios $\alpha \leq 4$. Consequently, we adopt $\alpha = 4$ for all subsequent experiments. We empirically verified that this setting remains effective at larger scales as well (*i.e.*, 95B total and 8B active).

Impact of number of experts. In Section 3, we noted that parameter reduction via compression can impede training stability. To quantify this, we pretrain the ℓ -MoE_{eff} LatentMoE variant of the 16B total and 2B active parameter model with the hidden dimension d compressed by a factor of 4, both with and without a compensatory increase in the total number of experts, using the baseline hyperparameters. Figure 3 shows that reducing d without scaling the expert count leads to significant quality degradation, validating the expert scaling strategy employed by LatentMoE.

Comparison between the two variants of LatentMoE. In Section 3, we introduced two LatentMoE variants: ℓ -MoE_{eff}, designed to improve inference efficiency while maintaining baseline accuracy, and ℓ -MoE_{acc}, designed to enhance accuracy at a comparable inference cost. Figure 3 compares the validation loss of these configurations against the baseline for the 16B total and 2B active parameter model using a latent dimension of $\ell = 512$ ($\alpha = 4$). Consistent with our expectations, ℓ -MoE_{eff} matches the baseline accuracy, whereas ℓ -MoE_{acc} achieves a noticeably lower validation loss. *We recommend ℓ -MoE_{acc} for Pareto-optimal accuracy versus inference cost.*

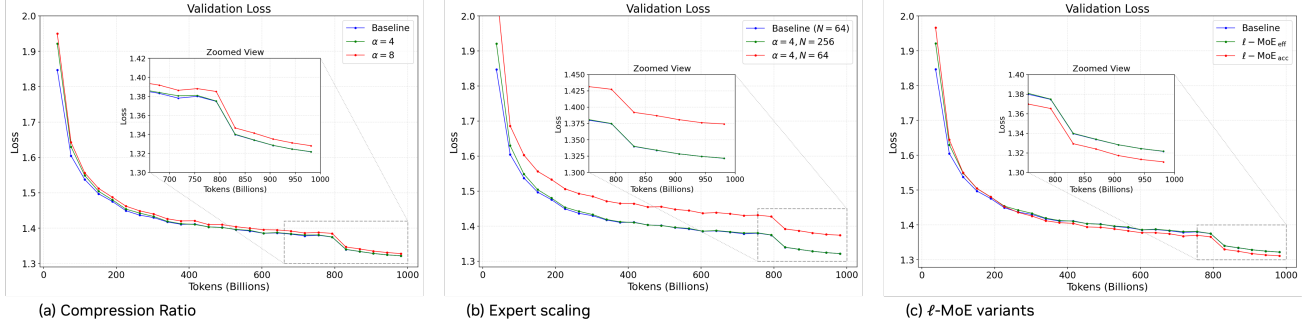


Figure 3. LatentMoE ablations on the 16BT-2BA model. *Left:* Sweeping the compression ratio $\alpha = d/\ell$ under the ℓ -MoE_{eff} configuration shows model quality is preserved up to $\alpha = 4$. *Middle:* Under $4\times$ compression, proportionally scaling the routed expert count ($N' = \alpha N$) recovers baseline training behavior, while omitting expert scaling leads to significant degradation. *Right:* Comparison between the standard MoE baseline and the two LatentMoE variants at $\ell = 512$. ℓ -MoE_{eff} matches baseline convergence, while ℓ -MoE_{acc} achieves consistently lower validation loss at comparable inference cost.

Table 4. Accuracy comparisons of the 95BT-8BA model with and without LatentMoE. Compared to the baseline, with equivalent parameters, ℓ -MoE_{acc} provides higher accuracy across all downstream tasks, while ℓ -MoE_{eff} provides comparable to or better accuracy with much fewer FLOPs.

Model	Active Params	Total Params	MMLU Pro	MMLU	Code	Math	Commonsense
Baseline	8.47B	94.4B	29.26	58.95	40.33	64.39	74.32
ℓ -MoE _{acc}	8.44B	94.8B	34.91	62.23	41.50	64.88	75.18
ℓ -MoE _{eff}	5.62B	94.8B	34.75	61.06	40.68	63.61	73.72

4.2. LatentMoE Scaling Studies

Leveraging the insights from the 16B model ablations, we train a 95B parameter Transformer using a LatentMoE configuration with a $4\times$ compression ratio. Figure 4 presents the validation loss trajectories for ℓ -MoE_{eff} and ℓ -MoE_{acc} relative to the baseline. Consistent with the 16BT-2BA results, ℓ -MoE_{eff} matches the baseline, while ℓ -MoE_{acc} demonstrates superior results. Table 4 shows the downstream task accuracy at the 300B token horizon. We report Code as the average over HumanEval, HumanEval+, MBPP, and MBPP+, Math as the average of GSM8K CoT and MATH-500, and Commonsense understanding as the average of RACE, ARC-Challenge, HellaSwag, and Winogrande. For simplicity, we used the same hyperparameters optimized for the baseline Transformer for LatentMoE. Further hyperparameter tuning might lead to better accuracy.

To further validate the effectiveness of the LatentMoE architecture, we also pretrain a series of hybrid Mamba-Attention MoE models. Specifically, we first train a baseline 8B active (73B total) parameter model. As described in Table 3, each MoE layer in the hybrid architecture contains 128 experts, 6 activated experts, 2 shared experts, and uses an intermediate FFN dimension of 2688. We use Squared-ReLU activation and a model dimension of 4096. We then train the ℓ -MoE_{eff} and ℓ -MoE_{acc} LatentMoE variants of the baseline model,

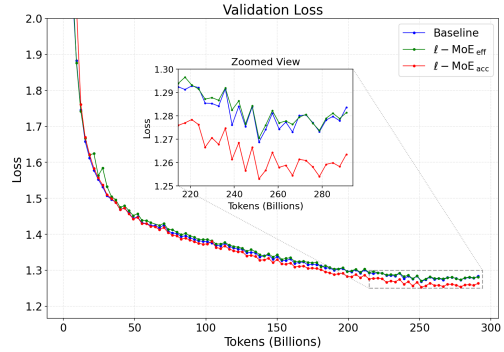


Figure 4. 95B Model Training Convergence. Validation loss curves for the 95BT-8BA baseline, ℓ -MoE_{eff}, and ℓ -MoE_{acc} configurations ($\ell = 1024$, $\alpha = 4$). ℓ -MoE_{eff} matches baseline convergence, while ℓ -MoE_{acc} outperforms the baseline.

Table 5. Accuracy comparisons of hybrid Mamba-Attention MoEs with and without LatentMoE. Compared to the baseline, with equivalent parameters, ℓ -MoE_{acc} provides higher accuracy across all downstream tasks, while ℓ -MoE_{eff} provides comparable or better accuracy with much fewer FLOPs.

Model	Active Params	Total Params	MMLU Pro	MMLU	Code	Math	Commonsense
Baseline	8.09B	72.6B	48.30	70.10	51.95	78.32	81.73
ℓ -MoE _{acc}	8.02B	72.8B	52.87	72.11	55.14	80.19	82.10
ℓ -MoE _{eff}	5.91B	72.8B	51.29	71.34	53.13	77.01	80.78

using a $4\times$ compression ratio.

Results after training the above models on 1T tokens are shown in Table 5. All models are trained with identical hyperparameters. As shown, the LatentMoE ℓ -MoE_{acc} variant achieves higher accuracy than the baseline across all tasks, while the ℓ -MoE_{eff} variant achieves accuracy comparable to or better than the standard granular MoE baseline.

Overall, the LatentMoE provides a clear advantage in terms of accuracy per FLOP and per parameter compared to standard MoEs, paving the way for higher accuracy at fixed inference cost or lower inference cost at fixed accuracy.

Table 6. Measured throughput (Tokens/s/GPU) for LatentMoE and Standard MoE across different concurrencies.

Model	BS=1	BS=4	BS=16	BS=64	BS=128
LatentMoE (Tokens/s/GPU)	181.6	528.5	1130.8	1569.6	1625.8
Standard MoE (Tokens/s/GPU)	206.6	509.8	1204.6	1549.3	1725.9

4.3. Inference Performance

ℓ -MoE_{acc} is expected to achieve similar inference speed to the standard MoE baseline while attaining higher accuracy. Our evaluations in Section 4.2 confirmed that ℓ -MoE_{acc} indeed achieves higher accuracy compared to standard MoE. Here, we evaluate ℓ -MoE_{acc} from the perspective of inference efficiency. Table 6 presents the measured performance of ℓ -MoE_{acc} compared to standard MoE for the Hybrid-73BT-8BA model (see Table 3) on two Hopper H100 GPUs using vLLM with FP8 per-tensor quantization. We focus our measurements on the hybrid Mamba-Attention baseline, as it represents the most efficient inference architecture.

The measurements show that at higher concurrencies, per-GPU throughput drops by only up to 6%. Further software optimizations could be performed to mitigate even these small throughput differences between LatentMoE and standard MoE. One proposed optimization is to utilize separate CUDA streams for routed and shared experts, which could reduce end-to-end latency when performing inference with smaller batches or when model dimensions do not saturate the GPU’s compute. A second optimization targets the MoE kernels from the CUTLASS library. Since LatentMoEs decrease the size of the GEMMs for routed experts, it is important to ensure that inner dimensions remain large enough to fully utilize the GPU and avoid SM-bound workloads. When inner dimensions are small, specialized smaller-matrix GEMM kernels should be used.

4.3.1. PROJECTED SERVING IMPACT AT TRILLION-PARAMETER SCALE

Inference efficiency can be characterized as a three-dimensional trade-off surface, with accuracy along one axis, throughput per GPU along a second axis, and latency (user interactivity) along the third axis. Thus far, we have discussed the accuracy of LatentMoE and presented measured performance at the 95B-parameter scale. In the following section, we examine a two-dimensional slice of this trade-off at the trillion-parameter scale by projecting throughput per GPU and latency frontiers for accuracy-matched models.

Performance evaluation methodology. We use a high-fidelity proprietary simulator to project end-to-end serving for a trillion-parameter model and its LatentMoE variant, evaluating 200K operating points to estimate Pareto frontiers in Figure 5. We model two traffic regimes: decode-heavy setting modeled with chunked piggybacking and

prefill-heavy setting modeled with disaggregated serving, selecting the serving strategy following (Mitra et al., 2025).

Effective Parameter Multiplier (EPM). We use the effective parameter multiplier to construct an iso-accuracy baseline for inference comparisons. We benchmark the native KIMI-K2-1T against our proposed variant, KIMI-K2-1T-LATENTMOE. Following the “Effective Parameter Count” framework established by (Frantar et al., 2025), we posit that a treated model with physical parameters N_{treat} behaves like a standard dense baseline with effective parameters N_{eff} . We assume baseline performance follows a scaling law $f(N)$. For a treated model achieving score S_{treat} , the effective capacity is obtained by inverting the baseline scaling law:

$$N_{eff} = f^{-1}(S_{treat}) \quad (3)$$

The EPM is defined as the ratio of effective capacity to physical parameters:

$$\lambda = \frac{N_{eff}}{N_{treat}} \quad (4)$$

We use λ to construct an iso-accuracy baseline with parameter count $N_{iso} = \lambda \cdot N_{treat}$. For our evaluations, we derive $f(N)$ by fitting a log-linear function to the MMLU accuracy scores of the QWEN-3-DENSE model family (0.6B, 1.7B, 4B, 8B, 14B, and 32B):

$$f(N) = a \cdot \log N + b \quad (5)$$

where a and b are fitted parameters.

Iso-accuracy baseline construction. We estimate an Effective Parameter Multiplier of $\lambda \approx 1.35\times$ for KIMI-K2-1T-LATENTMOE. For a 1T-parameter base model, this implies an iso-accuracy scale of $N_{iso} \approx 1.35T$, corresponding to an increase of $(1.35 - 1.0)T \approx 0.35T \approx 350B$ parameters. Guided by this target, we construct a physical **iso-accuracy baseline**, denoted KIMI-K2-1.35T, by scaling the native architecture depth from 61 to 80 layers. This construction matches the projected effective capacity implied by LatentMoE and enables a direct inference-efficiency comparison against a standard model of comparable predictive power.

Accuracy matching with standard MoE scaling is more expensive. ℓ -MoE_{acc} achieves an accuracy gain at fixed parameter and FLOP budget. When we enforce an accuracy-matched comparison using the standard MoE architecture, the required scaling incurs a marked serving penalty. Across the projected Pareto frontier (Figure 5), KIMI-K2-1.35T is approximately $1.24\times$ – $3.46\times$ slower than KIMI-K2-1T-LATENTMOE, indicating that ℓ -MoE_{acc} provides a more favorable accuracy-latency trade-off than increasing model size through the standard MoE architecture to reach the same accuracy target.

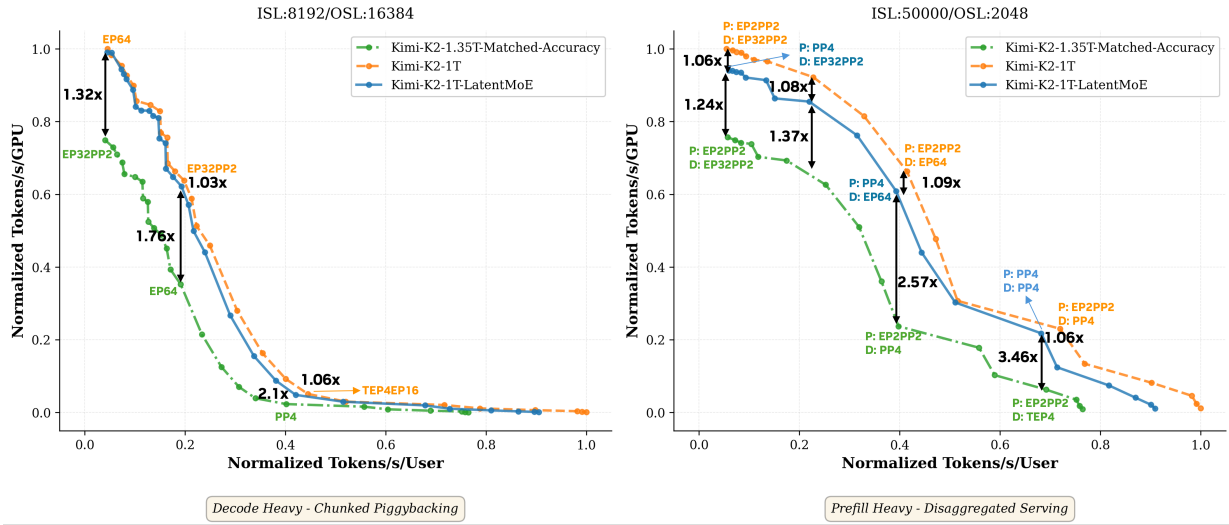


Figure 5. Projected throughput-latency Pareto frontiers at trillion-parameter scale. *Left:* Decode-heavy traffic (chunked piggy-backing). *Right:* Prefill-heavy traffic (disaggregated serving). KIMI-K2-1T-LATENTMOE attains accuracy-efficient operating points. Matching its accuracy with standard MoE scaling requires $\approx 350\text{B}$ extra parameters (KIMI-K2-1.35T), yielding a $1.24\times$ – $3.46\times$ projected slowdown. Representative operating points are annotated with their parallelism configuration, where EP denotes expert parallelism and PP pipeline parallelism (e.g., EP32PP2 is expert-parallel degree 32 with pipeline-parallel degree 2). In the disaggregated serving setting (*right*), the prefix P/D indicates the prefill and decode configurations, respectively.

Table 7. Simulator fidelity at the HYBRID-550BT scale. Measured vs. simulator-projected speedup of HYBRID-550BT-LATENTMOE over (i) the same-size standard MoE and (ii) an iso-accurate standard MoE that is $1.23\times$ larger (HYBRID-677BT).

Comparison	Source	BS=1	BS=64	BS=128	BS=256
vs. HYBRID-550BT (iso-param)	Measured	$0.87\times$	$1.00\times$	$1.03\times$	$1.02\times$
	Projected	$0.90\times$	$0.97\times$	$0.96\times$	$0.96\times$
vs. HYBRID-677BT (iso-accuracy)	Measured	$1.16\times$	$1.24\times$	$1.28\times$	$1.38\times$
	Projected	$1.11\times$	$1.19\times$	$1.18\times$	$1.18\times$

Simulator validation against measured silicon. We validate the simulator against measured throughput on HYBRID-550BT, a 550B total / 56B active hybrid Mamba-Attention model—the largest we can run end-to-end on available hardware. Applying the EPM construction at this scale yields $\lambda \approx 1.23$, giving an iso-accurate standard MoE baseline of $\approx 677\text{B}$ (HYBRID-677BT). Table 7 compares measured vs. projected throughput-per-GPU speedup of HYBRID-550BT-LATENTMOE over HYBRID-550BT (iso-param) and HYBRID-677BT (iso-accuracy), across BS 1–256. Mean error is $\approx 6\%$ ($\approx 15\%$ worst case, BS=256), and the simulator under-predicts LatentMoE’s advantage in 7 of 8 cells—the 1T projections in Figure 5 are therefore a conservative lower bound.

Latent projection overhead is modest. The latent projection adds two matrices per layer ($W_{\downarrow} \in \mathbb{R}^{\ell \times d}$, $W_{\uparrow} \in \mathbb{R}^{d \times \ell}$), totaling $2d\ell$ parameters, $4d\ell$ FLOPs, and $2d\ell$ elements of weight-load volume per token per layer. Relative to the routed expert computation ($4K'm\ell$ FLOPs across FC1 and

FC2 of all $K' = \alpha K$ active experts), this is a fraction $d/(K'm)$; for the 95B model ($\alpha=4$, $K'=24$, $m=2688$, $d=4096$) it evaluates to $\approx 6.35\%$. These analytical estimates are consistent with measured end-to-end overheads: on the 73B Hybrid Mamba-Attention model on H100 with FP8 (Table 6), LatentMoE incurs a $\approx 12\%$ throughput penalty at BS=1, which narrows to within $\approx 6\%$ at higher concurrencies; in our trillion-parameter projections, native KIMI-K2-1T remains within $\approx 9\%$ of KIMI-K2-1T-LATENTMOE. In all cases, the projection overhead is small compared to the cost of achieving the same accuracy via standard scaling to KIMI-K2-1.35T.

5. Related Work

Mixture-of-Experts (MoE) models have become a cornerstone of state-of-the-art large language models. We challenge the original MoE design paradigm and introduce an alternative architecture that achieves higher accuracy under iso-parameter and iso-FLOP constraints.

A broad literature reduces inference cost through model compression such as quantization (Rouhani et al., 2023b;a) and sparsity (Xie et al., 2024). At the expert level, prior work proposes pruning (Lu et al., 2024; Lasby et al., 2025; Chen et al., 2022) and merging (Li et al., 2024). These approaches are largely orthogonal to LatentMoE and can be composed with it.

The closest related work is MoLAE (Liu et al., 2025), which applies post-training low-rank approximation to ex-

pert weights in a latent space. While both leverage latent representations, they make fundamentally different trade-offs. LatentMoE couples latent-space routing with increased expert diversity and combinatorial sparsity to improve accuracy at fixed serving cost. MoLAE instead mitigates accuracy loss from latent projection using grouped projections and by restricting compression to a subset of expert components (FC2), which limits communication savings during dispatch and reduces memory bandwidth benefits. As discussed in Section 2, efficient MoE serving is typically not FLOP-bound, so reducing FLOPs alone is insufficient to improve the accuracy–throughput–latency Pareto frontier.

BigMac (Jin et al., 2025) also projects tokens to a lower dimension before dispatch, but differs in how the freed budget is reinvested. BigMac enlarges the FFN intermediate dimension and reports baseline-comparable accuracy, whereas LatentMoE scales the total and active expert counts and surpasses the baseline (e.g., +5.65 MMLU Pro at 95B; Table 4). Crucially, enlarging the intermediate dimension does not reduce per-expert memory bandwidth (Design Principle I), so latency-bound serving sees no improvement; LatentMoE instead reduces per-expert memory footprint by a factor α . Finally, LatentMoE is validated at 95B parameters and 1T tokens; BigMac is reported at GPT-3 6.7B scale.

LatentMoE is also complementary to two orthogonal lines of work. Multi-head Latent Attention (MLA) (DeepSeek-AI et al., 2024) applies low-rank projections to compress the KV-cache in attention layers, targeting attention-side memory bandwidth rather than LatentMoE’s expert-weight bandwidth and all-to-all communication. Concurrent work on Manifold-Constrained Hyper-Connections (mHC) (Xie et al., 2026) modifies residual connectivity rather than the expert path, widening the residual stream with learned, stability-constrained mixing. Both can be combined with LatentMoE in the same model.

6. Discussion

Implications for MoE design. Every major deployed MoE today runs expert computation at the full hidden dimension d , but our analysis shows this is not necessary. The expert dimension can be reduced and the freed budget reinvested into more total and active experts at the same inference cost. LatentMoE is the first architecture to make this trade-off explicit, and the accuracy gains in Tables 4 and 5 quantify what conventional MoE design leaves on the table. More broadly, the path to higher model quality does not require more training compute or larger clusters, but better architecture: at trillion-parameter scale, a 1T LatentMoE matches a 1.35T standard MoE, eliminating 350B parameters of serving cost through design alone. The roofline-guided framework behind these choices — identify the binding bottleneck, derive an architectural lever that

resolves it without hurting quality, and validate at scale — is not specific to LatentMoE; it provides a reusable template for evaluating future MoE modifications.

Deployment and environmental impact. Because LatentMoE matches or improves accuracy at iso-parameter and iso-FLOP budgets, practitioners do not need to scale model size or training compute to reach a given accuracy target. This directly reduces the GPU count required to serve frontier-quality models and lowers the barrier for organizations without large-scale clusters. It also reduces per-query energy and carbon footprint at scale: matching the accuracy of KIMI-K2-1.35T requires only a 1T LatentMoE, eliminating the energy cost of serving 350B extra parameters on every query.

Generality. The roofline-guided design principles in Section 2 are not specific to any hardware platform; as accelerator memory bandwidth continues to lag behind compute scaling, architectures that minimize data movement will become increasingly relevant. LatentMoE is similarly not specific to Transformers: our results on the 73B Mamba-Attention hybrid (Table 5) show it applies equally to hybrid models combining attention with alternative sequence-modeling layers such as Mamba.

Limitations. Our trillion-parameter projections rely on a high-fidelity simulator and on EPM-based accuracy extrapolation. The log-linear MMLU fit underlying the iso-accuracy baseline operates in the non-saturating regime, which makes the 1.35T estimate a conservative lower bound rather than a point estimate; the reported Pareto advantage is therefore not overstated, though the absolute gap will depend on realized scaling behavior. On the implementation side, LatentMoE’s smaller GEMM inner dimensions can require kernel tuning, a near-term barrier to adoption. Our current open-source implementation in vLLM¹ is already within $\approx 6\%$ of standard MoE throughput, and LatentMoE has been adopted in a publicly released frontier model (NVIDIA et al., 2026).

7. Conclusion

We introduced LatentMoE, a revised MoE architecture that improves accuracy per FLOP and per parameter by reducing routed dimension via latent projection, cutting all-to-all communication and expert weight bandwidth. The saved budget is reinvested into greater expert diversity at fixed inference cost. Across experiments up to 95B parameters, hybrid models, and projected trillion-parameter serving, LatentMoE improves the accuracy–efficiency Pareto frontier over standard MoE scaling.

¹<https://github.com/vllm-project/vllm/pull/30203>

Impact Statement

LatentMoE reduces the inference cost of large-scale Mixture-of-Experts language models by lowering memory bandwidth and all-to-all communication, while improving accuracy at fixed serving cost. As LLM inference becomes a substantial and growing fraction of datacenter energy consumption, the resulting reductions in GPU count, memory traffic, and communication translate into lower per-query energy use and carbon footprint at scale (Section 6), and broaden access to capable models for organizations and researchers without large compute budgets. At the same time, making trillion-parameter models significantly cheaper to deploy lowers the barriers for both beneficial and potentially harmful uses: the same efficiency gains that broaden beneficial access also reduce the cost of deploying capable models for misuse, generation of misleading content, and other risks well documented in the LLM literature. We believe this dual-use trade-off warrants continued attention from the safety and policy communities alongside ongoing efficiency work. Beyond this, we do not foresee impacts unique to this architecture beyond those already discussed for large language models in general.

References

- Barron, A. R. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945, 1993.
- Chen, T., Huang, S., Xie, Y., Jiao, B., Jiang, D., Zhou, H., Li, J., and Wei, F. Task-Specific Expert Pruning for Sparse Mixture-of-Experts, 2022. URL <https://arxiv.org/abs/2206.00277>.
- Dai, D., Deng, C., Zhao, C., Xu, R. X., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., Xie, Z., Li, Y. K., Huang, P., Luo, F., Ruan, C., Sui, Z., and Liang, W. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *CoRR*, abs/2401.06066, 2024. URL <https://arxiv.org/abs/2401.06066>.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., Sun, X., Wang, X., Liu, X., Xie, X., Yu, X., Song, X., Zhou, X., Yang, X., Lu, X., Su, X., Wu, Y., Li, Y. K., Wei, Y. X., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Zheng, Y., Zhang, Y., Xiong, Y., Zhao, Y., He, Y., Tang, Y., Piao, Y., Dong, Y., Tan, Y., Liu, Y., Wang, Y., Guo, Y., Zhu, Y., Wang, Y., Zou, Y., Zha, Y., Ma, Y., Yan, Y., You, Y., Liu, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Huang, Z., Zhang, Z., Xie, Z., Hao, Z., Shao, Z., Wen, Z., Xu, Z., Zhang, Z., Li, Z., Wang, Z., Gu, Z., Li, Z., and Xie, Z. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, 2024. URL <https://arxiv.org/abs/2405.04434>.
- Frankle, J. and Carbin, M. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJl-b3RcF7>.
- Frantar, E., Evci, U., Park, W., Houlshby, N., and Alistarh, D. Compression Scaling Laws: Unifying Sparsity and Quantization, 2025. URL <https://arxiv.org/abs/2502.16440>.
- Jin, Z., Wang, S., Zhu, J., Zhan, H., Bai, Y., Zhang, L., Ming, Z., and Li, C. BigMac: A Communication-Efficient Mixture-of-Experts Model Structure for Fast Training and Inference. In Walsh, T., Shah, J., and Kolter, Z. (eds.), *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*, pp. 17689–17698. AAAI Press, 2025. doi: 10.1609/AAAI.V39I17.33945. URL <https://doi.org/10.1609/aaai.v39i17.33945>.
- Lasby, M., Lazarevich, I., Sinnadurai, N., Lie, S., Ioannou, Y., and Thangarasa, V. REAP the Experts: Why Pruning Prevails for One-Shot MoE compression, 2025. URL <https://arxiv.org/abs/2510.13999>.
- Li, P., Zhang, Z., Yadav, P., Sung, Y.-L., Cheng, Y., Bansal, M., and Chen, T. Merge, Then Compress: Demystify Efficient SMoE with Hints from Its Routing Policy, 2024. URL <https://arxiv.org/abs/2310.01334>.
- Liu, Z., Wu, H., She, R., Fu, X., Han, X., Zhong, T., and Yuan, M. MoLAE: Mixture of Latent Experts for Parameter-Efficient Language Models, 2025. URL <https://arxiv.org/abs/2503.23100>.

- Lu, X., Liu, Q., Xu, Y., Zhou, A., Huang, S., Zhang, B., Yan, J., and Li, H. Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models, 2024. URL <https://arxiv.org/abs/2402.14800>.
- Mitra, T., Borkar, R., Bhatia, N., Matas, R., Raj, S., Mudigere, D., Zhao, R., Golub, M., Dutta, A., Madduri, S., et al. Beyond the Buzz: A Pragmatic Take on Inference Disaggregation. *arXiv preprint arXiv:2506.05508*, 2025.
- Novak, R., Bahri, Y., Abolafia, D. A., Pennington, J., and Sohl-Dickstein, J. Sensitivity and Generalization in Neural Networks: an Empirical Study. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=HJC2SzZCW>.
- NVIDIA, :, Blakeman, A., Grattafiori, A., Basant, A., Gupta, A., Khattar, A., Renduchintala, A., Vavre, A., Shukla, A., Bercovich, A., Ficek, A., Shaposhnikov, A., Kondratenko, A., Bukharin, A., Milesi, A., Taghibakhshi, A., Liu, A., Barton, A., Mahabaleshwarkar, A. S., Klein, A., Zuker, A., Geifman, A., Shen, A., Bhiwandiwalla, A., Tao, A., Agrusa, A., Verma, A., Guan, A., Mandarwal, A., Mehta, A., Aithal, A., Poojary, A., Ahamed, A., Mishra, A., Thekkumpate, A. K., Dattagupta, A., Zhu, B., Sadeghi, B., Simkin, B., Lanir, B., Schifferer, B., Nushi, B., Kartal, B., Rouhani, B. D., Ginsburg, B., Norick, B., Soubasis, B., Kisananin, B., Yu, B., Catanzaro, B., del Mundo, C., Hwang, C., Wang, C., Hsieh, C.-P., Zhang, C., Yu, C., Mungekar, C., Patel, C., Alexiuk, C., Parisien, C., Neale, C., Meurillon, C., Mosk-Aoyama, D., Su, D., Corneil, D., Afrimi, D., Lo, D., Rohrer, D., Serebrenik, D., Gitman, D., Levy, D., Stosic, D., Mosallanezhad, D., Narayanan, D., Nathawani, D., Rekesh, D., Yared, D., Kakwani, D., Ahn, D., Riach, D., Stosic, D., Minasyan, E., Lin, E., Long, E., Long, E. P., Segal, E., Lantz, E., Evans, E., Ning, E., Chung, E., Harper, E., Tramel, E., Galinkin, E., Pounds, E., Briones, E., Bakhturina, E., Tsykunov, E., Ladhak, F., Wang, F., Jia, F., Soares, F., Chen, F., Galko, F., Sun, F., Siino, F., Agam, G. H., Ajjanagadde, G., Bhatt, G., Prasad, G., Armstrong, G., Shen, G., Batmaz, G., Nalbandyan, G., Qian, H., Sharma, H., Ross, H., Ngo, H., Hum, H., Sahota, H., Wang, H., Soni, H., Upadhyay, H., Mao, H., Nguyen, H. C., Nguyen, H. Q., Cunningham, I., Galil, I., Shahaf, I., Gitman, I., Loshchilov, I., Schen, I., Levy, I., Moshkov, I., Golan, I., Putterman, I., Kautz, J., Scowcroft, J. P., Casper, J., Mitra, J., Glick, J., Chen, J., Oliver, J., Zhang, J., Zeng, J., Lou, J., Zhang, J., Choi, J., Huang, J., Conway, J., Guman, J., Kamalu, J., Greco, J., Cohen, J., Jennings, J., Daw, J., Vialard, J. V., Yi, J., Parmar, J., Xu, K., Zhu, K., Briski, K., Cheung, K., Luna, K., Wyss, K., Santhanam, K., Shih, K., Kong, K., Bhardwaj, K., Shankar, K., Puvvada, K. C., Pawelec, K., Anik, K., McAfee, L., Sleiman, L., Derczynski, L., Ding, L., Wei, L., Liebenwein, L., Vega, L., Grover, M., Segbroeck, M. V., de Melo, M. R., Nazemi, M., Sreedhar, M. N., Kilaru, M., Ashkenazi, M., Romeijn, M., Chochowski, M., Cai, M., Kliegl, M., Moosaei, M., Kulka, M., Novikov, M., Samadi, M., Corpuz, M., Wang, M., Price, M., Andersch, M., Boone, M., Evans, M., Martinez, M., Khona, M., Chrzanowski, M., Lee, M., Dabbah, M., Shoeybi, M., Patwary, M., Mulepati, N., Nabwani, N., Hereth, N., Assaf, N., Habibi, N., Zmora, N., Haber, N., Sessions, N., Bhatia, N., Jukar, N., Pope, N., Ludwig, N., Tajbakhsh, N., Ailon, N., Juluru, N., Sharma, N., Hrinchuk, O., Kuchaiev, O., Delalleau, O., Olabiyi, O., Argov, O. U., Puny, O., Tropp, O., Xie, O., Chadha, P., Shamis, P., Gibbons, P., Molchanov, P., Morkisz, P., Dykas, P., Jin, P., Xu, P., Januszewski, P., Thombre, P. P., Varshney, P., Gundecha, P., Tredak, P., Miao, Q., Wan, Q., Mahabadi, R. K., Garg, R., El-Yaniv, R., Zilberstein, R., Shafipour, R., Harang, R., Izzo, R., Shahbazyan, R., Garg, R., Borkar, R., Gala, R., Islam, R., Hesse, R., Waleffe, R., Watve, R., Koren, R., Zhang, R., Hewett, R., Hewett, R. J., Prenger, R., Timbrook, R., Mahdavi, S., Modi, S., Krizan, S., Lim, S., Kariyappa, S., Satheesh, S., Kaji, S., Pasumarthi, S., Muralidharan, S., Narentharen, S., Narenthiran, S., Bak, S., Kashirsky, S., Poulos, S., Mor, S., Ramasamy, S., Acharya, S., Ghosh, S., Sreenivas, S. T., Thomas, S., Fan, S., Gopal, S., Prabhunoye, S., Pachori, S., Toshniwal, S., Ding, S., Singh, S., Sun, S., Ithape, S., Majumdar, S., Singhal, S., Sergienko, S., Alborghetti, S., Ge, S., Devare, S. D., Barua, S. K., Panguluri, S., Gupta, S., Priyadarshi, S., Akter, S. N., Bui, T., Ene, T.-D., Kong, T., Do, T., Blankevoort, T., Moon, T., Balough, T., Asida, T., Natan, T. B., Ronen, T., Konuk, T., Vashishth, T., Karpas, U., De, U., Noorozi, V., Noroozi, V., Srinivasan, V., Elango, V., Cui, V., Korthikanti, V., Rao, V., Kurin, V., Lavrukhin, V., Anisimov, V., Jiang, W., Ahmad, W. U., Du, W., Ping, W., Zhou, W., Jennings, W., Zhang, W., Prazuch, W., Ren, X., Karnati, Y., Choi, Y., Meyer, Y., Wu, Y.-F., Zhang, Y., Qin, Y., Lin, Y., Geifman, Y., Fu, Y., Subara, Y., Suhara, Y., Gao, Y., Moshe, Z., Dong, Z., Zhu, Z., Liu, Z., Chen, Z., and Yan, Z. NVIDIA Nemotron 3: Efficient and Open Intelligence, 2025. URL <https://arxiv.org/abs/2512.20856>.
- NVIDIA, :, Chandiramani, A., Blakeman, A., Olaoye, A., Gupta, A., Somasamudramath, A., Khattar, A., Adesoba, A., Renduchintala, A., Asif, A., Agrawal, A., Vavre, A., Kiswani, A., Padmakumar, A., Hotchandani, A., Shukla, A., Bercovich, A., Ficek, A., Shaposhnikov, A., Gronskiy, A., Kondratenko, A., Neefus, A., Steiner, A., Yang, A., Bukharin, A., Young, A., Hatamizadeh, A., Taghibakhshi, A., Galiautdinova, A., Liu, A., Kumar, A., Mahabaleshwarkar, A. S., Klein, A., Zuker, A., Geifman, A., Bhiwandiwalla, A., Subramaniam, A., Tao, A., Shrivastava,

- A., Agrusa, A., Srivastava, A., Verma, A., Guan, A., Shors, A., Chockalingam, A., Mandarwal, A., Ramani, A., Mehta, A., Jain, A., Venkatesan, A., Anoosheh, A., Aithal, A., Poojary, A., Ahamed, A., Mishra, A., Demiroz, A. S., Thekkumpate, A. K., Sohrabizadeh, A., Kaur, A., Dattagupta, A., Anandan, B. S., Sadeghi, B., Simkin, B., Lanir, B., Schifferer, B., Chislett, B., Nushi, B., Kartal, B., Thiede, B., Rouhani, B. D., Chen, B., Ginsburg, B., Norick, B., Kisacanin, B., Yu, B., Catanzaro, B., Mani, B., del Mundo, C., Lee, C., Kim, C., Hwang, C., Ni, C., Wang, C., Truong, C., Hsieh, C.-P., Yu, C., Luo, C., Wang, C., Mungekar, C., Patel, C., Alexiuk, C., Holguin, C., Wing, C., Munley, C., Parisien, C., Desai, C., Sheng, C., Neale, C., Meurillon, C., Kumar, D., Gil, D., Su, D., Corneil, D., Afrimi, D., Triana, D. B. E., Egert, D., Fatade, D., Lo, D., Rohrer, D., Serebrenik, D., Sorokin, D., Gitman, D., Levy, D., Stosic, D., Edelson, D., Messina, D., Mosallanezhad, D., Tamok, D., Donia, D., Narayanan, D., O’Kelly, D., Peri, D., Nathawani, D., Wu, D., Rekesh, D., Yared, D., Kakwani, D., Tuttle, D. K. B., Ahn, D., Jiang, D., Poorkay, D., O’Flaherty, D., Riach, D., Stosic, D., Stee, D. V., Minasyan, E., Lin, E., Long, E. P., Segal, E., Lantz, E., Lewis, E., Evans, E., Ning, E., Chung, E., Harper, E., Pham-Hung, E., Tramel, E. W., Galinkin, E., Pounds, E., Etrog, E., Briones, E., Wu, E., Bakhturina, E., Tsykunov, E., Dobrowolska, E., Movahed, F. S., Memarian, F., Wang, F., Jia, F., Soares, F., Frujeri, F. V., Chen, F., Lin, F., Galko, F., Zhang, F., Siino, F., Hou, F., Bhatt, G., Prasad, G., Venkataramani, G., Gupta, G., Armstrong, G., Shen, G., Borghesi, G., Neskovic, G., Batmaz, G., Lam, G., Wu, G., Pauloski, G., Davis, G., Nalbandyan, G., Zhang, G., Farber, G., Huang, G., Qian, H., Kumar, H. K. S., Kim, H., Sharma, H., Iso, H., Ross, H., Hum, H., Sahota, H., Wang, H., Soni, H., Upadhyay, H., Nguyen, H., Cunningham, I., Galil, I., Shahaf, I., Padovani, I., Gitman, I., Shovkun, I., Dhillon, I., Loshchilov, I., Kelly, I., Schen, I., Levy, I., Moshkov, I., Golan, I., Putterman, I., Tu, J., Baczek, J., Kautz, J., Scowcroft, J. P., Rosenberg, J., Casper, J., Pflum, J., Grant, J., Sewall, J., Mitra, J., Glick, J., Chen, J., Oliver, J., Xu, J., Zhu, J., Song, J., Zhang, J., Zeng, J., Lou, J., Milton, J., Chow, J., Zhang, J., Choi, J., Huang, J., Huang, J., Caruso, J., Conway, J., Guman, J., Jatko, J., Kamalu, J., Greco, J., Cohen, J., Raiman, J., Jennings, J., Daw, J., Yu, J., Tapia, J., Yi, J., Parmar, J., Achar, J., Briski, K., Mattoo, K., Cheung, K., Luna, K., Wyss, K., Shih, K., Kong, K., Nguyen, K., Bhardwaj, K., Buryak, K., Sivamani, K. S., Krommydas, K., Murphy, K., Puvvada, K. C., Pawelec, K., Anik, K., Tewari, L., Sleiman, L., Du, L., Derczynski, L., Ding, L., Ilan, L., Wu, L., Wei, L., Vega, L., Su, L., Segbroeck, M. V., de Melo, M. R., Zhang, M., Fathi, M., Sreedhar, M. N., Sreedhar, M., Chandran, M. T., Gomez, M. R., Ashkenazi, M., Cuevas, M., Romeijn, M., Zhang, M., Cai, M., Gabel, M., Kliegl, M., Patelka, M., Moosaei, M., Varacalli, M., Novikov, M., Ferrato, M., Samadi, M., Corpuz, M., Xin, M., Wang, M., Wang, M., Price, M., Schaffer, M., Andersch, M., Boone, M., Evans, M., Wang, M. Z., Martinez, M., Khona, M., Chrzanowski, M., Hollinger, M., Ma, M., Lee, M., Dabbah, M., Shoeybi, M., Patwary, M., Mulepati, N., Khalil, N., Nabwani, N., Agarwal, N., Balasubramaniam, N., Hennouni, N., Kodukula, N., Hereth, N., Pinckney, N., Assaf, N., Habibi, N., Qin, N., Zmora, N., Haber, N., Reamaroon, N., Quak, N., Bhatia, N., Jukar, N., Pope, N., Ludwig, N., Tajbakhsh, N., Ailon, N., Juluru, N., De, N., Pitt, N., Rybakov, O., Hrinchuk, O., Kuchaiev, O., Delalleau, O., Olabiyi, O., Argov, O. U., Almog, O., Puny, O., Tropp, O., Padovani, O., Xie, O., Chadha, P., Shamis, P., Gibbons, P., Molchanov, P., Belcak, P., Jin, P., Xu, P., Januszewski, P., Jannaty, P., Shevate, P., Thalasta, P., Thombre, P. P., Varshney, P., Gambhir, P., Gundecha, P., Tredak, P., Miao, Q., Wan, Q., Minh, Q. T., Mahabadi, R. K., Oberman, R., Garg, R., Kandu, R., Zhong, R., El-Yaniv, R., Zilberstein, R., Shafipour, R., Yao, R., Pi, R., Mazzaresse, R., Wang, R., Izzo, R., Singla, R., Shahbazyan, R., Garg, R., Borkar, R., Gala, R., Islam, R., Clark, R., Hesse, R., Waleffe, R., Kalidindi, R. V., Watve, R., Koren, R., Fan, R., Kharwar, R., Cai, R., Zhang, R., Hewett, R. J., Prenger, R., Timbrook, R., Egashira, R., Mahdavi, S., Joshi, S. S. A., Modi, S., Krizan, S., Pombr, S., Kariyappa, S., Satheesh, S., Pombo, S., Kaji, S., Pasumarthi, S., Mishra, S., Muralidharan, S., Hara, S., Narenthiran, S., Rogawski, S., Na, S., Bak, S., Sameni, S., Poulos, S., Mor, S., Acharya, S., Lord, S. G. A., Sreenivas, S. T., Kotek, S., Gharghabi, S., Thomas, S., Lin, S.-C., Likhite, S., Fan, S., Chen, S., Gopal, S., Prabhunoye, S., Pachori, S., Toshniwal, S., Zhang, S., Ding, S., Renjith, S., Prayaga, S., Jain, S., Sun, S., Rella, S., Das, S., Ithape, S., S. S. H., Majumdar, S., Singhal, S., Singudasu, S. H., Niverty, S., Sergienko, S., Gloginic, S., Alborghetti, S., Ge, S., McCullough, S., Devare, S. D., Velury, S. V., Rao, S., Barua, S. K., Gai, S., Panguluri, S., Koundinyan, S., Patnam, S., Priyadarshi, S., Bhendigeri, S., Akter, S. N., Arunagiri, S., Yuan, T., Abramovich, T., Bui, T., Yu, T., Kong, T., Do, T., Gburek, T., Marques, T., Moore, T., Blankevoort, T., Moon, T., Ma, T., Mitra, T., Grzegorzec, T., Asida, T., Natan, T. B., Keren, T., Ronen, T., Rebedea, T., Starkey, T., Konuk, T., Vashishth, T., Condensa, T., Karpas, U., De, U., Noorozi, V., Noroozi, V., Shah, V. A., Vaidyanathan, V., Srinivasan, V., Elango, V., Cui, V., Korthikanti, V., Mehta, V., Adams, V., Wu, V., Kurin, V., Lavrukhin, V., Anisimov, V., Seo, W., Jiang, W., Ahmad, W. U., Du, W., Ping, W., Chen, W.-M., Quan, W., Dai, W., Gao, W., Jennings, W., Zhang, W., Ren, X., Xin, X., Li, X., Yu, Y., Chen, Y., Galron, Y., Karnati, Y., Choi, Y., Meyer, Y., Wu, Y.-F., Zhang, Y., Lin, Y., Geifman, Y., Fu, Y., Suhara, Y., Kwon, Y., Zhang, Y., Huang, Y., Moshe, Z., Wang, Z., Cheng, Z., Zhu, Z., Yang, Z., Liu,

- Z., Chen, Z., Yan, Z., and Ahmed, Z. Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning, 2026. URL <https://arxiv.org/abs/2604.12374>.
- NVIDIA Corporation. NVIDIA Blackwell Architecture Datasheet. <https://nvdam.widen.net/s/wnsxxrh2w/blackwell-datasheet-3384703>, 2024. Accessed: 2025.
- Rouhani, B. D., Zhao, R., Elango, V., Shafipour, R., Hall, M., Mesmakhosroshahi, M., More, A., Melnick, L., Golub, M., Varatkar, G., Shao, L., Kolhe, G., Melts, D., Klar, J., L’Heureux, R., Perry, M., Burger, D., Chung, E., Deng, Z. S., Naghshineh, S., Park, J., and Naumov, M. With Shared Microexponents, A Little Shifting Goes a Long Way. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA ’23, New York, NY, USA, 2023a. Association for Computing Machinery. ISBN 9798400700958. doi: 10.1145/3579371.3589351. URL <https://doi.org/10.1145/3579371.3589351>.
- Rouhani, B. D., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., Dusan, S., Elango, V., Golub, M., Heinicke, A., James-Roxby, P., Jani, D., Kolhe, G., Langhammer, M., Li, A., Melnick, L., Mesmakhosroshahi, M., Rodriguez, A., Schulte, M., Shafipour, R., Shao, L., Siu, M., Dubey, P., Micikevicius, P., Naumov, M., Verrilli, C., Wittig, R., Burger, D., and Chung, E. Microscaling Data Formats for Deep Learning, 2023b. URL <https://arxiv.org/abs/2310.10537>.
- Taylor, R., Ojha, V., Martino, I., and Nicosia, G. Sensitivity Analysis for Deep Learning: Ranking Hyper-parameter Influence. In *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 512–516, 2021. doi: 10.1109/ICTAI52525.2021.00083.
- Wang, L., Gao, H., Zhao, C., Sun, X., and Dai, D. Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts, 2024. URL <https://arxiv.org/abs/2408.15664>.
- Xie, Y., Zhang, Z., Zhou, D., Xie, C., Song, Z., Liu, X., Wang, Y., Lin, X., and Xu, A. MoE-Pruner: Pruning Mixture-of-Experts Large Language Model using the Hints from Its Router, 2024. URL <https://arxiv.org/abs/2410.12013>.
- Xie, Z., Wei, Y., Cao, H., Zhao, C., Deng, C., Li, J., Dai, D., Gao, H., Chang, J., Yu, K., Zhao, L., Zhou, S., Xu, Z., Zhang, Z., Zeng, W., Hu, S., Wang, Y.,
- Yuan, J., Wang, L., and Liang, W. mHC: Manifold-Constrained Hyper-Connections, 2026. URL <https://arxiv.org/abs/2512.24880>.